Contents

Scoring & Logic Documentation **2**
Overview . . . 2
Stage 1: Hybrid Matching Engine (MatchEngine.js) . . . 2
1.1 Lexical Scoring (BM25F) . . . 2
1.2 Semantic Scoring (Cosine Similarity) . . . 2
1.3 Structural Bonus Scoring . . . 2
1.4 Score Fusion . . . 3
1.5 Must-Have Gate (Critical Skill Enforcement) . . . 3
1.6 Deterministic Explainability . . . 3
Stage 2: Agentic Interest Assessment (ConversationEngine.js) . . . 4
2.1 Two-LLM Conversation System . . . 4
2.2 Judge LLM Scoring . . . 4
2.3 Self-Healing & Resilience . . . 4
Stage 3: Combined Scoring . . . 4
3.1 Final Score Fusion . . . 4
3.2 Re-ranking . . . 4
Production-Ready Features . . . 5
1. Serverless Architecture . . . 5
2. Deterministic Explainability . . . 5
3. Self-Healing AI . . . 5
4. Must-Have Gate . . . 5
File Locations . . . 5

Architecture Diagram **6**
Component Overview . . . 6
User Interface Layer . . . 6
Document Processing Layer . . . 6
AI Extraction Layer . . . 6
Matching Engine Layer . . . 6
Interest Assessment Layer . . . 6
Data Layer . . . 6
Data Flow . . . 8

Scoring & Logic Documentation

Overview

Agentic Recruitment uses a sophisticated multi-stage scoring system that combines lexical matching, semantic similarity, structural bonuses, and AI-powered interest assessment to rank candidates. The system is designed to be deterministic, explainable, and production-ready.

Stage 1: Hybrid Matching Engine (MatchEngine.js)

1.1 Lexical Scoring (BM25F)

Purpose: Match exact skill keywords and domain terms from the job description.

Algorithm: BM25F (Best Matching 25 with Field weighting)

- Tokenizes JD and candidate profiles into normalized terms
- Builds an inverted index over candidate skills, experience, and domain knowledge
- Calculates document frequency (DF) and inverse document frequency (IDF) for each term
- Applies field-specific weights (skills: 1.0, experience: 0.8, domain: 0.6)

Formula:

```
BM25F(D, Q) = sum IDF(qi) x (f(qi, D) x (k1 + 1)) / (f(qi, D) + k1 x (1 - b + b x |D|/avgdl))
```

Where: - `f(qi, D)` = frequency of query term in document - `|D|` = document length - `avgdl` = average document length in corpus - `k1` = term saturation parameter (1.5) - `b` = length normalization parameter (0.75)

Output: Raw BM25 score (0-inf), normalized to 0-100 across corpus.

1.2 Semantic Scoring (Cosine Similarity)

Purpose: Match conceptual similarity beyond exact keywords (e.g., "React" <-> "Frontend").

Algorithm: Cosine similarity between embedding vectors

- Generates 768-dimensional embedding vectors using Gemini's `gemini-embedding-2` model
- Computes cosine similarity between JD embedding and candidate embedding
- Captures semantic relationships and context

Formula:

```
cosine_sim(A, B) = (A . B) / (||A|| x ||B||)
```

Output: Similarity score (0-1), scaled to 0-100.

1.3 Structural Bonus Scoring

Purpose: Reward candidates who meet practical constraints.

Components:

Factor	Weight	Logic
Experience	+15 max	Proximity to required years (+/-1 year = full points)
Location	+10 max	Exact match = full, same country = 50%, remote = 75%
Salary	+10 max	Within 20% of JD range = full points
Availability	+5 max	Immediate availability = full points

Formula:

```
bonusScore = (expBonus x 0.15) + (locBonus x 0.10) + (salBonus x 0.10) + (availBonus x 0.05)
```

1.4 Score Fusion

Purpose: Combine all signals into a single match score.

Weights: - BM25F (Lexical): 60% - Cosine Similarity (Semantic): 25% - Structural Bonuses: 15%

Formula:

```
rawFusedScore = (bm25Score x 0.50) + (vectorScore x 0.35) + (bonusScore x 0.15)
```

1.5 Must-Have Gate (Critical Skill Enforcement)

Purpose: Prevent false positives from candidates with many "nice-to-have" skills but missing core "must-have" requirements.

Algorithm: Mathematical hard-gate based on must-have skill coverage.

Formula:

```
mustHaveRatio = matchedMustHaves / totalMustHaves
mustHaveGate = 0.35 + 0.65 x mustHaveRatio
finalMatchScore = rawFusedScore x mustHaveGate
```

Behavior: - If candidate has 0% must-have skills: `mustHaveGate = 0.35` (max score clamped to 35%) - If candidate has 100% must-have skills: `mustHaveGate = 1.0` (no penalty) - Linear interpolation between these extremes

Example: - Candidate A: 10 nice-to-have skills, 0 must-have skills -> Max score: 35/100 - Candidate B: 5 nice-to-have skills, 3/3 must-have skills -> Max score: 100/100

1.6 Deterministic Explainability

Purpose: Generate human-readable explanations without LLM calls (zero latency, zero hallucination).

Algorithm: Set-diffing and mathematical operations.

Process: 1. Tokenize JD must-have and nice-to-have skills 2. Perform set intersection with candidate skills -> `matchedSkills` 3. Perform set difference -> `missingSkills` 4. Calculate experience proximity, salary alignment, location match 5. Generate "strength tags" based on score components

Output: - `must_have_matched`: Array of matched required skills - `must_have_missing`: Array of missing required skills - `nice_to_have_matched`: Array of matched optional skills - `strength_tags`: Descriptive tags (e.g., "Strong technical fit", "Experience mismatch") - `summary`: Natural language summary of match quality

Advantages: - 0ms latency (pure math) - 0 API cost - 100% factual accuracy (no hallucinations)

Stage 2: Agentic Interest Assessment (ConversationEngine.js)

2.1 Two-LLM Conversation System

Purpose: Simulate recruiter-candidate conversations to assess genuine interest.

Architecture: - **Recruiter LLM:** Roleplays as recruiter, asks questions about role, company, compensation - **Candidate LLM:** Roleplays as candidate, responds based on their profile - **Judge LLM:** Analyzes conversation transcript to score interest

Conversation Flow: 1. Recruiter initiates with role introduction 2. Candidate responds with interest/questions 3. 5-8 turn dialogue covering: - Role fit and expectations - Company culture/values - Compensation negotiation - Availability and timeline 4. Judge LLM evaluates transcript

2.2 Judge LLM Scoring

Purpose: Extract structured interest assessment from conversation.

Output Schema:

```
{
  "interest_score": 85,        // 0-100 numerical score
  "interest_level": "high",    // "high", "medium", "low"
  "blockers": ["Salary expectations above range"],
  "positive_signals": ["Strong cultural fit", "Immediate availability"],
  "summary": "Candidate shows genuine interest with minor compensation concerns"
}
```

Scoring Criteria: - **Engagement quality:** Depth of questions, enthusiasm in responses - **Alignment:** Fit with role requirements, company values - **Flexibility:** Willingness to negotiate on compensation, timeline - **Proactivity:** Asking insightful questions, showing research

2.3 Self-Healing & Resilience

JSON Repair: - Uses `jsonrepair` library to fix malformed LLM JSON responses - Handles missing quotes, trailing commas, unescaped characters - Ensures reliable parsing of structured outputs

Exponential Backoff Retry: - Handles Gemini API rate limits (HTTP 429) - Retries with exponential backoff: 2ˆattempt x 500ms - Max 3 retries before failing - Provides clear error messages to user

---

Stage 3: Combined Scoring

3.1 Final Score Fusion

Purpose: Combine match quality with interest level for final ranking.

Formula:

```
combinedScore = (matchScore x 0.60) + (interestScore x 0.40)
```

Rationale: - Skills are necessary but not sufficient (60% weight) - Interest compensates for minor skill gaps (40% weight) - Reflects real-world hiring decisions

Example: - Candidate A: Match 90, Interest 40 -> Combined: 70 - Candidate B: Match 75, Interest 85 -> Combined: 79 - **Result:** Candidate B ranks higher despite lower match score

3.2 Re-ranking

Process: 1. Sort candidates by `combinedScore` (descending) 2. Break ties with `matchScore` (higher match wins) 3. Display final shortlist with both scores visible

---

Production-Ready Features

1. Serverless Architecture

- MatchEngine runs entirely in-memory (no external vector DB)
- BM25 inverted index built on client-side
- Cosine similarity computed with vanilla JavaScript
- Edge-ready: Can run on Cloudflare Workers, Vercel Edge, etc.

2. Deterministic Explainability

- Zero-latency explanations via set-diffing
- No LLM calls for match explanations
- 100% factual accuracy, no hallucinations

3. Self-Healing AI

- JSON repair for malformed LLM responses
- Exponential backoff for API rate limits
- Graceful error handling with user-friendly messages

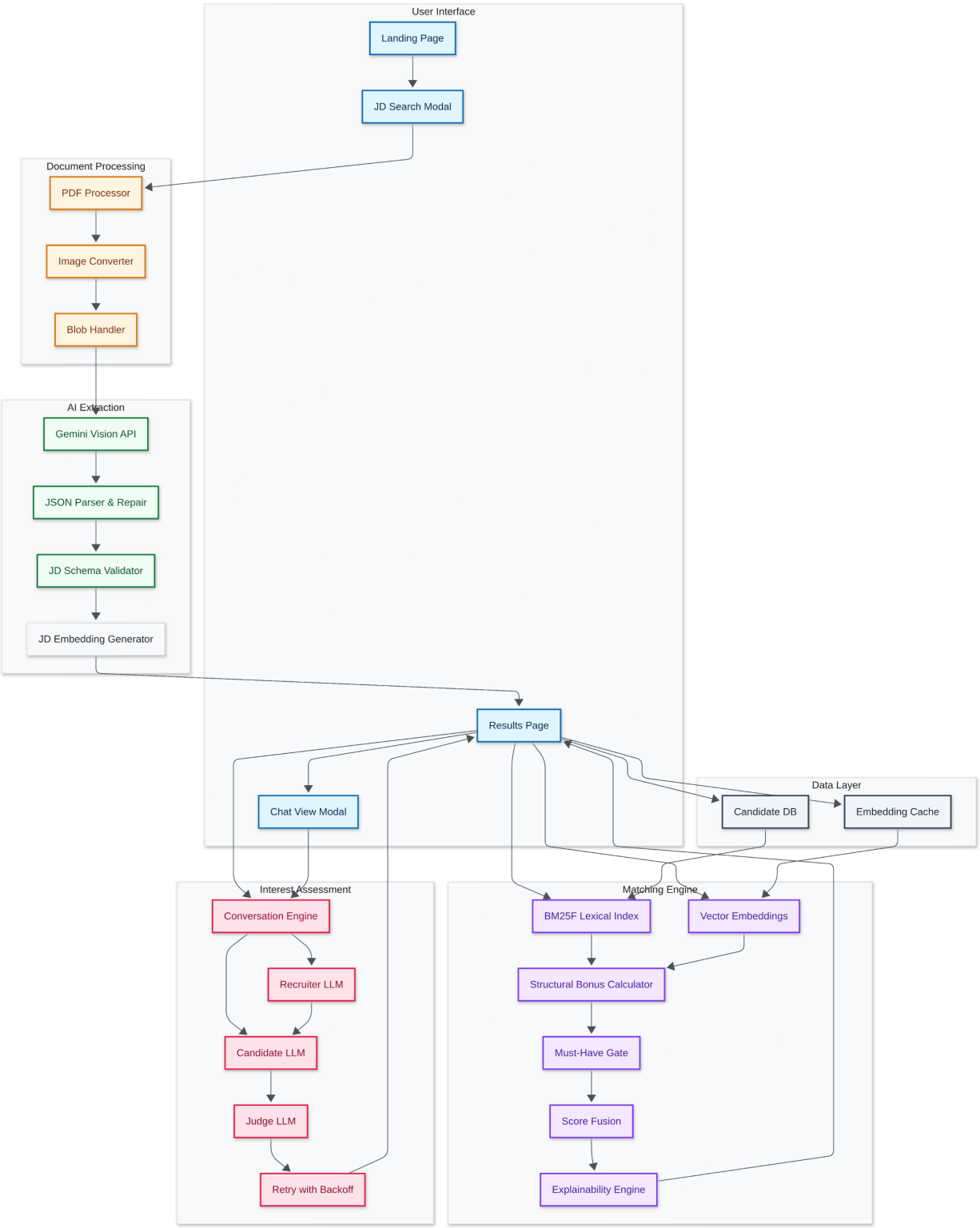
4. Must-Have Gate

- Mathematical enforcement of critical skills
- Prevents false positives from keyword stuffing
- Linear interpolation based on must-have coverage

File Locations

Component	File
Hybrid Matching	`src/utils/MatchEngine.js`
Interest Assessment	`src/utils/ConversationEngine.js`
Gemini Integration	`src/utils/buggu/geminiAI.js`
JSON Utilities	`src/utils/buggu/jsonUtils.js`
PDF Processing	`src/utils/buggu/pdfProcessor.js`
Embedding Generation	`src/utils/embeddingGenerator.js`

Architecture Diagram



Data Flow

1. **JD Upload**: User uploads PDF/image/text -> Modal
2. **Document Processing**: PDF -> Images -> Blobs
3. **AI Extraction**: Blobs -> Gemini -> Structured JSON
4. **Matching**: JD + Candidate DB -> BM25 + Vector + Bonuses -> Match Score
5. **Interest Assessment**: Selected candidates -> Two-LLM conversations -> Interest Score
6. **Combined Scoring**: Match Score (60%) + Interest Score (40%) -> Final Ranking
7. **Display**: Results page shows ranked candidates with scores and explanations